Workflow Lisp Frontend

Full Drain vs. Ralph Loop

Implementation quality, throughput, churn, and DSL semantics

PL frontend	Performance	Next semantics
Parsing, ASTs, type checks, macro/module semantics, and lowering into the existing workflow runtime.	Static comparison of full workflow drain coverage against the faster Ralph iteration loop.	Adjudicated providers as a step toward workflow-native judging and search loops.

Workflow DSL vocabulary

Workflow

A named graph of work. It declares shared context, available providers, and ordered or routed steps.

Step

One executable node in the workflow. A step may run a provider, command, assertion, call, loop, branch, or other DSL operation.

Provider

A reusable command template for an external worker, such as `codex exec`. Steps bind to providers and pass them prompts, files, and params.

In the Ralph loop

`DrainLispFrontendWork` is the repeat loop. `RunRalphIteration` is the Codex provider step. Each bundle reports `CONTINUE`, `DONE`, or `BLOCKED`, and the loop routes from that status.

Current Ralph runtime loop

```yaml
- name: DrainLispFrontendWork
  repeat_until:
    id: ralph_single_provider_iteration
    max_iterations: 500
    outputs:
      drain_status:
        from:
          ref: self.steps.RunRalphIteration.artifacts.drain_status
    condition:
      any_of:
        - compare:
            left: {ref: self.outputs.drain_status}
            op: eq
            right: DONE
        - compare:
            left: {ref: self.outputs.drain_status}
            op: eq
            right: BLOCKED
    on_exhausted:
      outputs:
        drain_status: BLOCKED
```

Runtime meaning

- `repeat_until` owns iteration count and stop condition.
- `CONTINUE` runs another `RunRalphIteration`.
- `DONE` and `BLOCKED` terminate the loop.
- Exhausting 500 iterations becomes `BLOCKED`.

The iteration step validates docs and injects paths

```yaml
- name: RunRalphIteration
  id: run_ralph_iteration
  provider: codex
  asset_file: prompts/lisp_frontend_autonomous_drain/ralph_iteration.md
  timeout_sec: 7200
  depends_on:
    required:
      - ${inputs.steering_path}
      - ${inputs.full_design_path}
      - ${inputs.mvp_design_path}
      - ${inputs.progress_ledger_path}
    optional:
      - ${inputs.backlog_root}/*.md
      - docs/plans/LISP-FRONTEND-AUTONOMOUS-DRAIN/**/*.md
      - docs/design/workflow_lisp_frontend*.md
    inject:
      mode: list
      position: prepend
      instruction: "Read these workspace files as needed..."
```

Runtime dependency contract

- Steering, full design, MVP design, and progress ledger must exist before an iteration runs.
- Optional backlog, plan, and design paths are injected when present.
- The prompt comes from an `asset_file`; it is not inline YAML prose.
- The bundle is validated before the repeat loop reads artifacts.

Ralph output bundle is the loop API

Example bundle

```
{
  "drain_status": "CONTINUE",
  "iteration_report_path": "artifacts/work/RALPH/iterations/
      iteration-3.md",
  "iteration_summary": "Implemented one bounded slice.",
  "selected_work_kind": "DESIGN_GAP",
  "selected_work_id": "parser-source-spans",
  "changed_files_path": "state/RALPH/drain/iterations/3/changed-
      files.json"
}
```

```
output_bundle:
  path: ${inputs.drain_state_root}/iterations/${loop.index}/ralph
      -iteration.json
  fields:
    - name: drain_status
      allowed: [CONTINUE, DONE, BLOCKED]
    - name: iteration_report_path
      under: artifacts/work
      must_exist_target: true
    - name: changed_files_path
      under: state
      required: false
```

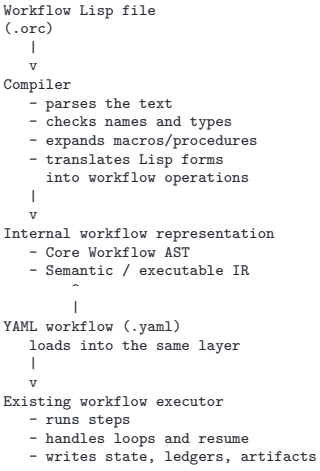
Contract

The output bundle, not free-form prose, drives loop control and artifact extraction.

Prompt policy

One bounded slice per iteration: choose work, implement, run focused verification, commit relevant files, write report and bundle.

Workflow Lisp as a PL frontend



Why not just a DAG?

Workflows are not just one-way task graphs. Steps can branch, retry, pause/resume, repair saved state, and commit outputs. Ralph is the simplest example: on success, it sends itself around again.

What the compiler produces

It does not emit machine code or rewrite the workflow as YAML. It builds checked workflow objects that the existing executor runs.

Why a Lisp frontend over YAML?

YAML: intent plus plumbing

```yaml
- name: ExecuteImplementation
  provider: codex
  asset_file: prompts/.../implement.md
  output_bundle:
    path: state/attempt.json

- name: SelectAttempt
  select_variant_output:
    path: state/attempt.json
    discriminant:
      name: implementation_state
      allowed: [COMPLETED, BLOCKED]
    variants:
      COMPLETED:
        fields:
          - name: execution_report_path

- name: PublishReport
  when: implementation_state == COMPLETED
  requires_variant:
    step: SelectAttempt
    value: COMPLETED
  command:
    - python
    - -c
    - |
      target = Path(sys.argv[1])
      pointer = Path(sys.argv[2])
      if not target.exists():
          raise SystemExit(2)
      pointer.write_text(target.as_posix() + "\n")
    - ${steps.SelectAttempt.artifacts.execution_report_path}
    - state/execution_report_path.txt
```

Lisp: type and branch together

```
;; ReportPath is a checked relpath type.
(defunion ImplementationAttempt
  (COMPLETED
    (execution-report-path ReportPath))
  (BLOCKED
    (progress-report-path ReportPath)
    (blocker-class BlockerClass)))

(let ((attempt
        ;; providers.execute is a build-time alias,
        ;; e.g. the codex provider in this workflow.
        (provider-result providers.execute
          :prompt prompts.implementation.execute
          :returns ImplementationAttempt)))
  (match attempt
    ;; completed is the local name for this case.
    ((COMPLETED completed)
      completed.execution-report-path)
    ((BLOCKED blocked)
      blocked.progress-report-path)))
```

Point

Same runtime target: provider output, variant selection, and a published report path. In Lisp, `defunion` declares the provider result type; each `match` arm names the selected case and exposes only that case's fields. Lowering emits the checked workflow objects that YAML authors currently spell out by hand.

Procedure first; macro only for syntax

`defproc`: reuse typed workflow logic

```
(defproc choose-report
  ((attempt ImplementationAttempt))
  -> ReportPath
  :effects ()
  :lowering inline
  (match attempt
    ((COMPLETED completed)
      completed.execution-report-path)
    ((BLOCKED blocked)
      blocked.progress-report-path)))
```

Use this when the inputs and outputs are already typed frontend values.

`defmacro`: generate source forms

```
(defmacro implementation-step
  (name provider prompt)
  (defworkflow name
    ((report_path ReportPath))
    -> ImplementationAttempt
    (provider-result provider
      :prompt prompt
      :inputs (report_path)
      :returns ImplementationAttempt)))

(implementation-step run-implementation
  providers.execute
  prompts.implementation.execute)
```

Use this only when the abstraction must create forms such as `defworkflow`, `provider-result`, or `match` before typechecking and lowering.

Rule

Macros are not the core claim. Typed authoring and checked lowering are. A macro expands to ordinary frontend syntax; the executor never runs the macro itself.

Compiler terms in plain language

Term	Meaning in this workflow frontend
Syntax forms	Parsed source shapes with enough structure to say what was written and where errors should point.
Frontend AST	The Lisp-specific tree after names and forms are understood as workflow concepts.
Typed AST	The frontend tree after the compiler checks records, unions, workflow inputs, return values, and allowed operations.
Lowering	The translation from Lisp-specific concepts into the shared workflow representation.
IR	Intermediate representation: the validated meaning the runtime can execute, resume, and persist.

Short version

The compiler turns human-authored Lisp into a checked workflow meaning. The executor then runs that meaning with the same state, ledger, artifact, and recovery machinery used by YAML workflows.

What the AST means

Authored Lisp shape

```
(defworkflow RalphDrain
  (repeat-until DrainLispFrontendWork
    (:max-iterations 500)
    (step RunRalphIteration
      (provider codex)
      (output-bundle ralph-iteration))))
```

Simplified tree shape

```
Workflow
  name: RalphDrain
  steps:
    RepeatUntil
      name: DrainLispFrontendWork
      max_iterations: 500
      body:
        Step
          name: RunRalphIteration
          run: Provider(codex)
          output_bundle: ralph-iteration
```

Point

The AST is not text to execute directly. It is the workflow split into named parts so the compiler can check it, transform it, and lower it into the runtime's workflow representation.

Full drain vs. Ralph loop: throughput

Metric	Full Lisp drain	Ralph loop
Elapsed wall time inspected	31.45 hours	12.85 hours
Completed units	12 design gaps	114 Ralph iterations
Raw rate	0.38 design gaps/hour	8.87 iterations/hour
Durable design gaps	12 recorded gaps	0 ledger-recorded gaps
MVP completion estimate	about 92%	about 55%
Full-design completion estimate	about 78%	about 25%
Frontend source LOC	20,756	7,856
Direct Lisp tests	277 tests	178 tests
`.orc` fixtures	86	133

Read the units carefully

The Ralph loop is much faster in raw iteration count, but the units are smaller. The full drain produces larger reviewed design-gap bundles and reaches much deeper into the full Workflow Lisp design.

Source: static inspection of run state, summaries, ledgers, artifacts, source, fixtures, and tests. No tests were run.

Implementation quality comparison

Metric	Full Lisp drain	Ralph loop
Full implementation quality	7.6 / 10	5.5 / 10
Equivalent subset quality	8.0 / 10	6.5 / 10
Spec adherence	8.0 / 10	5.5 / 10
Correctness evidence	7.2 / 10	5.8 / 10
Test coverage	8.6 / 10	6.5 / 10
Maintainability	6.7 / 10	5.8 / 10
Churn severity	7.0 / 10	8.0 / 10

Quality readout

The full drain is broader, more durable, and better covered by static evidence. The Ralph loop iterates much faster, but its subset remains less complete, less ledger-backed, and more concentrated in repeated edits to the same files.

Scores are static-inspection estimates, not test results. Higher quality scores are better; higher churn severity means more repeated repair/rework signal.

Progress quality: speed vs. churn

Full Lisp drain

- Slower but more complete.
- Covers parser, typing, lowering, macros, modules, procedures, stdlib, build artifacts, diagnostics, and migration pieces.
- Git history since the drain start shows the hottest source at 13 touches and the hottest test at 8 touches.
- The run has durable design-gap completion, but no per-iteration changed-file ledger.

Ralph loop

- Faster early iteration velocity.
- Best suited to small MVP slices and focused regression additions.
- Changed-file ledgers show concentrated repeat touches across lowering, expression validation, and related tests.
- Hot spots: `test_workflow_lisp_lowering.py` 61 touches; `lowering.py` 58 touches.

Repeated-file disparity

Ralph's hottest source file concentration remains much higher than the full-drain git-history proxy: 58 changed-file touches versus 13 source-file touches. This points to fast local iteration, but also to repeated stabilization in the same subset.

Concept: adjudicated provider

Current YAML surface

```yaml
- name: DraftDocument
  adjudicated_provider:
    candidates:
      - id: candidate_a
        provider: demo_candidate_a
      - id: candidate_b
        provider: demo_candidate_b
    evaluator:
      provider: demo_evaluator
      input_file: prompts/evaluate_candidate.md
      evidence_confidentiality: same_trust_boundary
    selection:
      tie_break: candidate_order
    score_ledger_path: artifacts/evaluations/scores.jsonl
  expected_outputs:
    - name: selected_document
      type: relpath
```

Conceptual Lisp surface

This form would live inside a surrounding `defworkflow` or `defproc`; it is not itself a top-level definition.

```
(adjudicated-provider draft-document
  :candidates
    ((candidate-a :provider providers.candidate-a)
     (candidate-b :provider providers.candidate-b))
  :evaluator providers.evaluator
  :evidence same-trust-boundary
  :tie-break candidate-order
  :returns SelectedDocument)
```

Runtime meaning

One baseline -> candidate workspaces -> output validation -> evaluator scores -> selected output promotion -> score ledger.

YAML exposes the protocol. A Lisp frontend can name the operation once and lower to the same validated candidate/evaluator/promotion machinery.

This is an early DSL-semantic step toward LLM-as-a-judge workflows: once judging is a typed operation, later search loops can evolve candidates and use adjudication as the fitness signal.

Toward self-hosted adjudication

Current v2.11 semantics

- The evaluator is a provider template plus evaluator prompt/rubric source.
- It returns strict score JSON: candidate id, finite score, and summary.
- It is not itself a nested workflow, `call`, `defproc`, or general DSL expression.

Self-hosted direction

- Treat the judge as a typed DSL unit: evidence packet in, score decision out.
- Candidate generation, judging, promotion, and score-ledger writing then live in one semantic layer.
- Genetic or search-style loops can evolve candidates while adjudication supplies the fitness function.

This keeps today's contract honest while showing the path from provider-level adjudication to workflow-native LLM-as-a-judge semantics.